

¿Qué sobrevive a una lectura equivocada? El solver como verificador de la percepción en KenKen desde imagen

Rody Sebastian Vilchez Marin (U202216562), José Javier Villanueva Aramburu (U202311894),
Jorge Rolando García Roca (U202211490)
Grupo 3 ù Tópicos en Ciencias de la Computación (IACC0058)
Universidad Peruana de Ciencias Aplicadas (UPC)
Lima, Perú

Resumen—Resolver un KenKen desde una imagen encadena dos fases distintas: percibir el tablero y razonar sobre él. Un error en la primera amenaza con volverse una solución falsa en la segunda. Construimos un sistema que separa ambas fases con un contrato explícito. La visión reconstruye tamaño, jaulas y etiquetas sin aprendizaje automático (umbral adaptativo, homografía, periodicidad de la rejilla, grosor de las aristas y correlación con plantillas). El razonamiento resuelve la instancia con un modelo de Programación con Restricciones en CP-SAT y comprueba si la solución es única. Evaluamos sobre 70 tableros reales de KrazyDad, de 4×4 a 9×9 , con la verdad extraída del texto vectorial de los propios PDF, y encontramos tres cosas. (1) La lectura acierta el 97.09 % de 895 etiquetas (κ de Cohen de 0.969) y el 96.45 % en los cuadernillos que no usamos para desarrollar. El error se concentra: 21 de los 26 fallos son un 5 leído como 6. (2) 48 tableros se leen bien de punta a punta, y ninguno de los otros 22 produce una solución errónea. De ellos, 3 se rechazan por una etiqueta ilegible; en 19 el solver demuestra que lo leído no tiene solución y, con restricciones reificadas, repara 16 de esas 19 sin ningún error silencioso. En un caso, el mismo mecanismo delató un error en la verdad extraída del PDF. (3) Un conjunto sintético sobreestima la precisión: lee bien el 97.8 % de los tableros limpios con tipografías conocidas, frente a 48 de 70 tableros reales. La propagación de las restricciones globales basta para resolver los 70 tableros y refutar las 19 lecturas sin solución, sin ramificar una sola vez. Con desigualdades binarias, los 9×9 necesitan hasta 5883 ramas. Resolver un tablero real leído tarda como máximo 20.4 ms. Separar percepción y razonamiento no solo resuelve el puzzle: convierte al solver en el control de calidad de lo que la visión cree haber leído.

Index Terms—KenKen, Programación con Restricciones, CP-SAT, Visión Computacional, OCR, verificación

I. INTRODUCCIÓN

KenKen es un cuadrado latino de $n \times n$ dividido en *jaulas*: grupos de celdas contiguas con una operación aritmética y un objetivo que sus valores deben cumplir. Resolverlo desde una foto exige dos cosas: percibir la estructura del tablero (tamaño, jaulas y etiquetas) y razonar sobre ella. Las dos fallan de manera distinta. Un solver de restricciones es exacto sobre la instancia que recibe; la visión, no. Si la visión lee un 5 donde hay un 6, cabría temer que el solver resuelva con exactitud el puzzle equivocado.

Separamos las dos fases con un contrato: la visión entrega una *instancia* (tamaño y lista de jaulas con sus celdas, opera-

ción y objetivo), y el razonamiento solo ve esa instancia. La separación tuvo un efecto que no buscábamos y que resultó ser el hallazgo central. Como un puzzle publicado tiene solución única, el solver reconoce cuándo lo que recibe no puede serlo. Aportamos tres cosas: (1) un pipeline de visión sin aprendizaje que detecta las jaulas por el grosor de las aristas; (2) un modelo de CP parametrizado en n que, gracias a sus restricciones globales, resuelve todos los tableros reales sin búsqueda, cuenta soluciones y, con restricciones reificadas, repara las lecturas sin solución; y (3) una evaluación sobre 70 tableros reales con verdad exacta, en la que ningún tablero mal leído llega a una solución errónea y un conjunto sintético resulta ser un mal juez de la precisión. La Fig. 1 resume la arquitectura: la app envía la foto a un nodo de cómputo, la visión la convierte en una instancia y el solver la resuelve y cuenta sus soluciones. Solo una solución única vuelve a la app.

II. TRABAJO RELACIONADO

II-0a. Percibir y razonar a la vez: Mulamba *et al.* [1], extendiendo [2], resuelven sudokus fotografiados con inferencia conjunta. Las probabilidades del clasificador entran al solver de restricciones, que elige la asignación más probable entre las que cumplen las reglas y así corrige errores de percepción. Muestran por qué hace falta: con un clasificador que acierta el 99 % de las celdas, predecir y luego resolver acierta el tablero completo solo en torno al 44 % de las veces. Es la base de la aplicación Sudoku Assistant de KU Leuven [3]. En la línea neurosimbólica, SATNet [4] integra un solver de satisfacibilidad diferenciable en una red. Pero Chang *et al.* [5] muestran que, sin etiquetas intermedias, no logra anclar los símbolos y su exactitud cae a cero.

II-0b. Dónde nos ubicamos: Nuestro sistema ocupa otro punto del espacio de diseño. Como los pipelines clásicos de sudoku por visión [7], anclamos los símbolos de forma explícita y determinista, con correlación normalizada [6] contra plantillas. Al solver le damos dos papeles: verificar la única lectura que entrega la visión y, si no tiene solución, repararla. A diferencia de la inferencia conjunta, la reparación no usa las probabilidades del clasificador, solo las confusiones de glifo

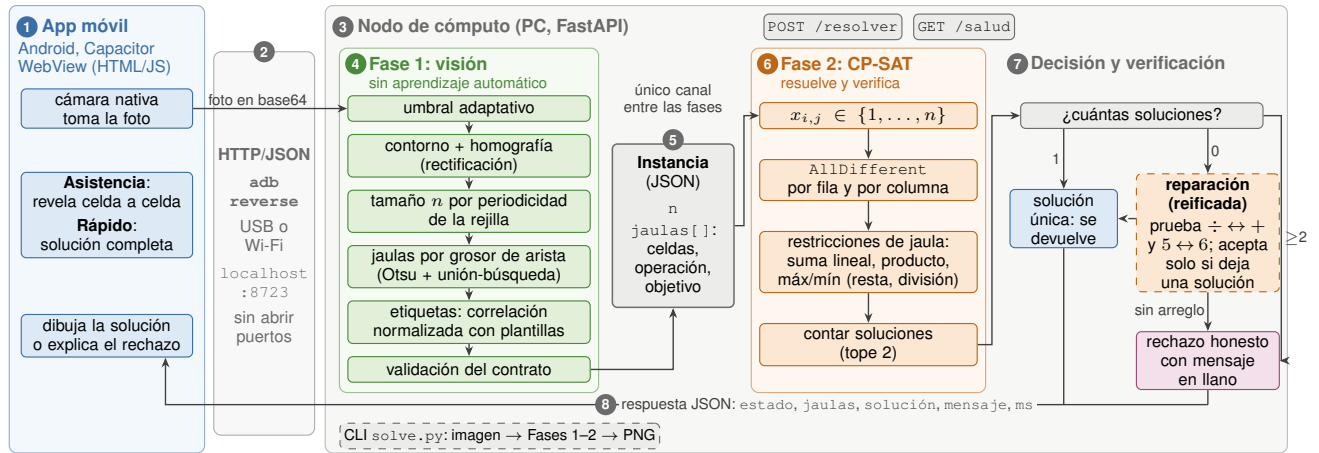


Figura 1. La visión y el solver solo se hablan a través de la instancia, y solo una solución única vuelve al usuario. (1) La app móvil toma la foto y la envía en JSON; (2) `adb reverse` la lleva por USB o Wi-Fi al puerto 8723 de la PC sin abrir puertos; (3) el nodo de cómputo atiende `POST /resolver`. (4) La Fase 1 reconstruye el tablero sin aprendizaje y (5) entrega la instancia, que es el contrato entre fases; (6) la Fase 2 la resuelve en CP-SAT y cuenta hasta dos soluciones; (7) con una sola, se devuelve; con ninguna, la reparación guiada por el solver (en trazo discontinuo) prueba confusiones típicas con restricciones reificadas y solo acepta una corrección que deje solución única; con varias, o si la reparación no encuentra una corrección inequívoca, el sistema rechaza con un mensaje en llano. (8) La respuesta vuelve a la app. El CLI `solve.py` recorre las mismas dos fases sobre una imagen.

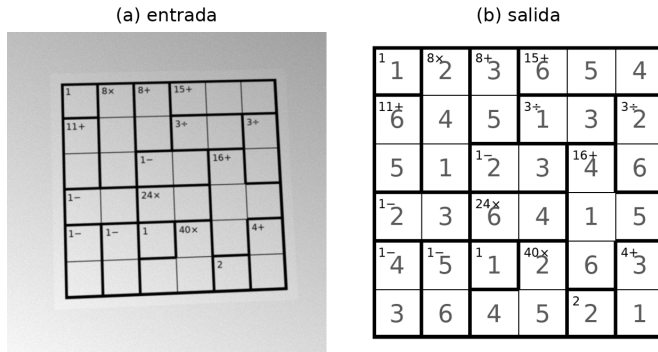


Figura 2. El sistema lee un tablero fotografiado con perspectiva, luz desigual y ruido, y devuelve la solución dibujada sobre una rejilla limpia. (a) Entrada: tablero sintético de 6×6 degradado como una foto. (b) Salida de `solve.py`: las 17 jaulas leídas sin error y su solución única.

que observamos. El rechazo ya basta para que ningún tablero mal leído llegue a una solución errónea (Sección VI-D). La reparación, además, recupera 16 de las 19 lecturas sin solución (Sección VI-E).

III. LA VERDAD SALE DEL PROPIO PDF

No encontramos un conjunto de datos público y etiquetado de tableros KenKen. El único precedente de KenKen por visión que hallamos (370 glifos reales) no declara licencia y estira los recortes a 28×28 , lo que destruye la proporción del glifo. Lo descartamos y trabajamos con dos fuentes propias.

III-0a. Tableros reales: Usamos cinco cuadernillos de KrazyDad [8]: 4×4 fácil y difícil, 6×6 fácil y difícil, y 9×9 difícil. Un extractor recorta cada tablero de la página, lo rectifica a 1000×1000 píxeles y recorta la etiqueta de cada jaula. Así obtenemos 72 tableros (48, 16 y 8 por tamaño) y 895 etiquetas. Como los PDF son vectoriales, cada etiqueta está como texto con su caja. Al proyectarla con la homografía del tablero obtenemos su celda y, con ella, una verdad exacta

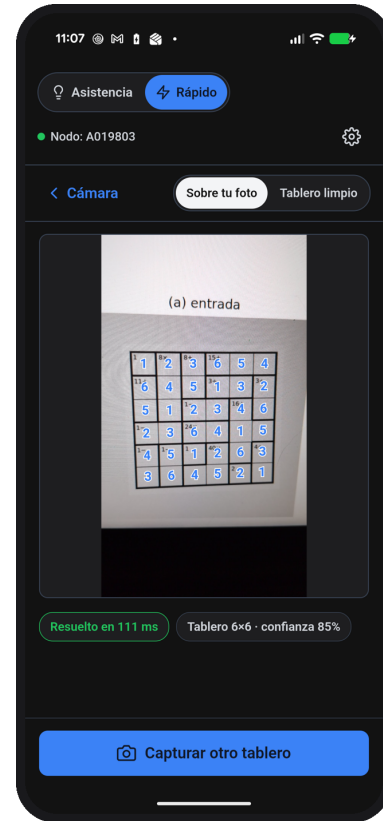


Figura 3. El sistema funciona en un teléfono real. Captura de pantalla de un Pixel 9a, sin retoques: la app fotografió un tablero mostrado en una pantalla y dibujó la solución sobre la propia foto, cada número en su celda pese a la inclinación (Modo Rápido, Sección VII). Leerlo y resolverlo tomó 111 ms.

sin etiquetar nada a mano. La celda sale de la geometría y no del detector de jaulas, así que el detector no se mide contra sí mismo. Como el extractor no produjo la verdad de dos tableros, la evaluación de punta a punta usa 70. Los primeros

experimentos (Tabla III) usaron un subconjunto inicial de 16 tableros de los cuadernillos difíciles de 6×6 y 9×9 , rasterizados a 150 ppp. KrazyDad autoriza el uso escolar y prohíbe el comercial. Los PDF se descargan bajo demanda y no se redistribuyen.

III-Ob. Tableros sintéticos: Un generador parte de un cuadrado latino aleatorio, lo divide en jaulas de hasta cuatro celdas, les asigna una operación compatible y dibuja el tablero con distintas tipografías, opcionalmente con perspectiva, iluminación desigual y ruido (Fig. 2a). Da verdad gratis y cubre condiciones que los PDF no tienen. Pero, como muestra la Sección VI-C, no basta para medir.

IV. FASE 1: DE LA IMAGEN A LA INSTANCIA, SIN APRENDIZAJE

IV-A. Localización y rectificación

Binarizamos la imagen en escala de grises con un umbral adaptativo, que tolera una iluminación desigual. Del contorno externo de mayor área obtenemos, por aproximación poligonal, un cuadrilátero cuyas esquinas definen una homografía que rectifica el tablero a un cuadrado [9]. Todo lo posterior trabaja sobre esa imagen rectificadora, nunca sobre la original.

IV-B. El tamaño se deduce de la periodicidad, no de líneas completas

En un KenKen impreso la línea fina se interrumpe donde la atraviesa una jaula, y un sesgo de medio grado la parte en tramos a distintas alturas. Por eso no podemos exigir que cada línea cruce el tablero. Lo que sí se conserva es la periodicidad. Para cada candidato n predecimos las líneas interiores de ambos ejes y puntuamos la tinta de la *más débil*: con el n correcto esa línea conserva alrededor del 29 % de su longitud en tinta, y con uno equivocado cae a cero. Elegimos el mayor n con todas sus líneas presentes. Los divisores del n correcto también pasan la prueba (una rejilla de 3 cae sobre la mitad de las líneas de un tablero de 6), pero solo la más fina las explica todas.

IV-C. Las jaulas se separan por el grosor de cada arista

Conocida la rejilla, sabemos dónde está cada arista interior y basta medir su grosor. Tomamos la mediana, a lo largo de la arista, del ancho del trazo de tinta *más cercano* a la posición prevista. La mediana descarta los cruces con otras líneas y con los dígitos. Buscar el trazo más cercano, en vez de exigir tinta en el píxel exacto, absorbe el error de la rectificación. Los grosores son bimodales porque el puzzle se imprime con dos anchos de pluma. El umbral entre fino y grueso sale del método de Otsu [10], con el marco exterior como referencia de grueso. Las celdas que no separa un borde grueso se agrupan con una estructura de unión-búsqueda.

IV-D. Las etiquetas se leen por correlación con plantillas

La etiqueta se imprime en la esquina superior izquierda de la celda *ancla* de la jaula, la primera en orden de fila y columna. El margen del recorte no es fijo: medimos dónde termina la línea de la rejilla y dejamos un respiro proporcional. Con un

margen fijo del 10 % de la celda se cortaba la parte superior de los dígitos (un 40 se leía 4U), y ese era el mayor error de todo el pipeline.

Separamos los caracteres por componentes conexas y re-agrupamos las que se solapan en horizontal, porque el signo $\div$ se imprime como tres manchas. Cada carácter se clasifica por correlación normalizada [6] contra plantillas renderizadas en siete tipografías, conservando la proporción del glifo. Durante el desarrollo, añadir rasgos estructurales, como el número de huecos cerrados para separar 3 de 8, *bajó* la precisión sintética de 99.4 % a 95.4 %. Lo que sí resolvió esas confusiones fue ampliar el banco de tipografías.

IV-E. Una lectura que viola el contrato no llega al solver

Antes de pasarla a la Fase 2 validamos la instancia: las jaulas deben particionar el tablero y ser conexas, y las de resta y división deben tener exactamente dos celdas. Una lectura que viola el contrato se rechaza aquí.

V. FASE 2: UN MODELO QUE RESUELVE Y VERIFICA

V-A. Variables, dominios y cuadrado latino

Sea X una matriz de variables enteras de $n \times n$ con

$$x_{i,j} \in \{1, 2, \dots, n\} \quad \forall i, j \in \{1, \dots, n\}.$$

Cada fila y cada columna es una permutación de $1..n$, que expresamos con la restricción global `AllDifferent`:

$$\begin{aligned} \text{ALLDIFFERENT}(x_{i,1}, \dots, x_{i,n}) \quad \forall i, \\ \text{ALLDIFFERENT}(x_{1,j}, \dots, x_{n,j}) \quad \forall j, \end{aligned}$$

$2n$ restricciones globales en total. El modelo se construye a partir de la instancia y no contiene ningún dato de un tablero concreto: el mismo código resuelve cualquier n y cualquier partición en jaulas.

V-B. Restricciones de jaula

Para cada jaula C_k con variables V_k , operación op_k y objetivo T_k :

- **Igualdad (=):** $v = T_k$, con $V_k = \{v\}$.
- **Suma (+):** $\sum_{v \in V_k} v = T_k$, una restricción lineal.
- **Producto ($\times$):** $\prod_{v \in V_k} v = T_k$, con la restricción global de multiplicación de CP-SAT.
- **Resta (-):** con $V_k = \{a, b\}$, $\text{máx}(a, b) - \text{mín}(a, b) = T_k$.
- **División ($\div$):** con $V_k = \{a, b\}$, $\text{máx}(a, b) = T_k \cdot \text{mín}(a, b)$.

En resta y división el orden en que la visión lista las dos celdas no significa nada, y el par $\text{máx} / \text{mín}$ hace el modelo simétrico. Cada una de esas jaulas añade dos variables auxiliares, $M_k = \text{máx}(a, b)$ y $m_k = \text{mín}(a, b)$, con dominio $1..n$. Para la división basta la igualdad del producto: con dominios enteros positivos implica que el cociente es exacto.

V-C. No hace falta reificar para resolver; sí para reparar

La formulación directa de la resta es una disyunción, $(a - b = T_k) \vee (b - a = T_k)$, que se modela reificando cada lado en una variable booleana: $\beta_k \Rightarrow a - b = T_k$ y $\neg\beta_k \Rightarrow b - a = T_k$ (en CP-SAT, `OnlyEnforceIf`). Lo mismo vale para la división. No la usamos: el par *máx/mín* expresa la misma relación sin variables booleanas, es simétrico en las dos celdas y ninguna otra restricción del modelo es condicional. Medimos las dos formulaciones (Sección VI-F) y rinden igual, así que la elección es de claridad, no de eficiencia.

La reificación sí es necesaria para reparar lecturas (Sección VI-E). Para cada jaula k consideramos su lectura y las alternativas a un solo glifo de distancia ($\div \leftrightarrow +$, $5 \leftrightarrow 6$, ...). Cada candidata c lleva un literal $\ell_{k,c}$, imponemos $\sum_c \ell_{k,c} = 1$ y activamos la restricción de la jaula con $\ell_{k,c} \Rightarrow \text{jaula}_k(c)$. El producto, el máximo y el mínimo se definen sin condición, porque CP-SAT no reifica esas restricciones; lo que se reifica es la igualdad con el objetivo. Minimizamos el número de lecturas cambiadas y, a igual número, la menos plausible según los fallos observados. Ese es el uso natural de la reificación en este problema: decidir qué lectura creer.

V-D. Contar soluciones convierte al solver en verificador

Resolvemos el modelo con CP-SAT [11], que combina propagación de restricciones con un núcleo SAT con aprendizaje de cláusulas. Además de buscar una solución, contamos soluciones con un tope de dos y detenemos la enumeración al llegar a él. Un puzzle publicado tiene solución única, así que una lectura que deja el tablero sin solución, o con más de una, delata un error de la Fase 1.

V-E. Exponencial en teoría, milisegundos en la práctica

El modelo tiene n^2 variables de dominio n , $2n$ restricciones `AllDifferent`, una restricción por jaula y dos variables auxiliares por cada jaula de resta o división: su tamaño es $O(n^2)$, lineal en el número de celdas.

Resolverlo es otra cosa. El espacio de asignaciones tiene n^{n^2} puntos, y las restricciones de cuadrado latino lo reducen al número de cuadrados latinos de orden n [12], que sigue creciendo de forma superexponencial: 576 para $n = 4$, unos 8.1×10^8 para $n = 6$ y unos 5.5×10^{27} para $n = 9$. Completar un cuadrado latino parcial ya es NP-completo [13], y para puzzles de esa familia con restricciones aritméticas por bloques, como `BlockSum`, también se ha probado NP-completitud [14]. No cabe esperar, entonces, una garantía de tiempo polinomial para `KenKen`. En la práctica, las jaulas de un puzzle publicado restringen tanto que la propagación basta: ningún tablero real requiere ramificar (Sección VI-F). Contar con tope dos cuesta, en el peor caso, una búsqueda completa adicional, porque demostrar que no hay una segunda solución, o ninguna, obliga a agotar el árbol.

VI. EVALUACIÓN

VI-A. El error de lectura se concentra en un par de glifos

Sobre las 895 etiquetas del conjunto real, la lectura completa (número y operación) coincide con la verdad en el

Tabla I
EL ERROR NO SE REPARTE: 21 DE LOS 26 FALLOS DE ETIQUETA SON UN 5 LEÍDO COMO 6

Verdad	Leído	Casos
5+	6+	17
15+, 15*, 35*	16+, 16*, 36*	4
2400*, 4800*, 2160*	ilegible	3
6*, 288*	8*, 286*	2
Total (de 895 etiquetas)		26

Tabla II
LOS CUADERNILLOS NO USADOS PARA DESARROLLAR RINDEN CASI IGUAL ($p = 0.24$) Y TAMPOCO PRODUCEN SOLUCIONES ERRÓNEAS; CONCENTRAN, ESO SÍ, EL FALLO DOMINANTE (16 DE SUS 17 ERRORES SON 5+ LEÍDO COMO 6+)

Conjunto	Etiquetas			Tableros			
	N	%	κ	N	Bien	Rech.	Sin sol.
Desarrollo (6H, 9H)	416	97.84	0.977	15	8	3	4
No usados (4E, 4H, 6E)	479	96.45	0.960	55	40	0	15
Total	895	97.09	0.969	70	48	3	19

97.09% de los casos. Como el reparto de etiquetas está muy desbalanceado, medimos también el acuerdo con el κ de Cohen [15], que descuenta el acuerdo esperable por azar. Obtenemos $\kappa = 0.969$, “casi perfecto” en la escala de Landis y Koch [16].

El error no se reparte (Tabla I). Cuatro de cada cinco fallos son la misma confusión, un 5 leído como 6 (21 de 26), y 17 de ellos caen en la etiqueta 5+, muy frecuente en los tableros de 4×4 . Los tres casos ilegibles son etiquetas de cuatro cifras que desbordan su celda y dejan el operador fuera del recorte. Los dos problemas, un par de glifos y un caso geométrico, admiten un arreglo dirigido.

VI-A0a. Desarrollo frente a cuadernillos no usados:

Ajustamos el recorte y el alfabeto de las plantillas mirando los 16 tableros iniciales, de los cuadernillos 6H y 9H, así que esas etiquetas no son un conjunto de prueba limpio. En los tres cuadernillos que no usamos para desarrollar (4E, 4H y 6E), la lectura acierta el 96.45% de 479 etiquetas ($\kappa = 0.960$), frente al 97.84% de los de desarrollo (Tabla II). La diferencia no es significativa (prueba exacta de Fisher, $p = 0.24$), pero el tipo de error cambia. En los cuadernillos no usados, 16 de los 17 fallos son 5+ leído como 6+, una confusión que en desarrollo casi no apareció. El modo de fallo dominante solo salió a la luz en datos que nadie miró al construir el sistema.

El mismo script, ejecutado en la máquina de otro integrante, dio 95.98% y $\kappa = 0.957$. La causa más probable es que el banco de plantillas se renderiza con las tipografías instaladas en cada máquina, aunque no la aislamos. Las cifras de este informe son las de la máquina descrita en la Sección VI-G.

VI-B. Plantillas y Tesseract empanan; los demás motores no leen operadores

Sobre el subconjunto inicial de 16 tableros, el mismo con que ajustamos el recorte y las plantillas, comparamos las plantillas con tres motores genéricos sobre las mismas 412

Tabla III

PLANTILLAS Y TESSERACT EMPATAN ($p = 0.065$); RAPIDOCR Y TROCR LEEN LOS NÚMEROS PERO NO LOS OPERADORES. EXACTITUD SOBRE LAS 412 ETIQUETAS DEL CONJUNTO DE DESARROLLO (%)

Método	Etiqueta	Operación	Número
Plantillas (este trabajo)	98.8	99.3	98.8
Tesseract 5.3.4	97.1	97.8	97.8
RapidOCR (PP-OCR)	64.8	68.4	94.2
TrOCR base printed	60.9	63.6	95.1

Tabla IV

EL CONJUNTO SINTÉTICO ES DE DESARROLLO: GUIÓ EL DISEÑO Y NO ESTIMA LA PRECISIÓN. FUERA DEL BANCO DE TIPOGRAFÍAS, LA INSTANCIA COMPLETA CAE AL 71.4 % EN FOTO. FASE 1 SOBRE 80 TABLEROS SINTÉTICOS (%)

Condición	Tipografía	N	Etiquetas	Instancia completa
Limpio	En el banco	45	99.9	97.8
Foto	En el banco	45	99.8	95.6
Limpio	Fuera del banco	35	99.4	91.4
Foto	Fuera del banco	35	98.2	71.4

etiquetas (Tabla III): Tesseract [17], RapidOCR (que ejecuta PP-OCR [18] sobre ONNX Runtime) y TrOCR [19]. Plantillas y Tesseract no se distinguen: discrepan en 11 etiquetas, 9 a favor de las plantillas y 2 de Tesseract ($p = 0.065$, prueba exacta de McNemar [20]). Los dos superan con claridad a RapidOCR y TrOCR ($p < 0.001$), que leen bien los números pero no los operadores. Más que el orden, importa que los dos mejores fallan en etiquetas distintas: al menos uno acierta 411 de las 412 etiquetas (99.8%), que es el techo de un jurado de motores. La comparación se hizo sobre el conjunto de desarrollo y, por eso, favorece a las plantillas: es una cota optimista de su ventaja, no una medida.

VI-C. El conjunto sintético es de desarrollo, y sobreestima

Con el código final, la Fase 1 lee bien el 97.8 % de los tableros sintéticos limpios con tipografías del banco y el 95.6 % de los degradados como foto (Tabla IV). Con tipografías fuera del banco cae al 91.4 % y al 71.4 %, aunque son clones de familias que el banco contiene. Estos tableros guiaron el diseño: son un conjunto de desarrollo, no una estimación de la precisión. Y sobreestiman: en los tableros reales, la instancia completa sale bien en 48 de 70.

La razón se vio al estrenar los tableros reales, cuando el pipeline falló en los 16 primeros. La rejilla real se interrumpe, la tinta no cae en el píxel predicho, un margen fijo cortaba los dígitos y KrazyDad imprime glifos que el banco no tenía. Son cuatro supuestos que el generador nunca puso a prueba.

VI-D. Ningún tablero mal leído produce una solución errónea

Pasamos cada uno de los 70 tableros con verdad por el sistema completo, cotejamos la instancia leída con la del PDF y resolvimos lo leído (Tabla V). El tamaño n es correcto en los 67 tableros que producen una instancia, y la partición en jaulas coincide con la verdad en 66. La excepción es el tablero cuya verdad resultó errónea, que discutimos abajo. En

Tabla V

DE LOS 22 TABLEROS MAL LEÍDOS, NINGUNO ENTREGA UNA SOLUCIÓN ERRÓNEA: 3 SE RECHAZAN Y EN 19 EL SOLVER DEMUESTRA QUE NO HAY SOLUCIÓN

n	Tableros	Bien leídos	Rechazados	Sin solución
4	47	36	0	11
6	16	9	1	6
9	7	3	2	2
Total	70	48	3	19

el subconjunto inicial a 150 ppp el detector fundía dos jaulas en 6 de 16 tableros; con la rasterización a 1000×1000 ya no ocurre. Los fallos que quedan son de etiqueta, los de la Tabla I.

En 3 de los 22 tableros mal leídos una etiqueta resulta ilegible y la instancia se rechaza antes del solver. En los otros 19 la instancia es válida, pero el solver demuestra que no tiene solución: un solo dígito mal leído bastó para que el cuadrado latino dejara de cerrar.

El mismo mecanismo encontró un error en la verdad. En un tablero de 9×9 la instancia extraída del PDF no tiene solución, mientras que la lectura del sistema tiene una única: el extractor no registró una jaula de división. Como un puzzle publicado tiene solución, el error está en la verdad y no en la lectura. Por eso ese tablero cuenta como bien leído en la Tabla V; contado de forma estricta, el total es 47 de 70. Con 70 tableros no es una garantía, pero el modelo de CP verifiqué tanto la lectura como los datos con que la evaluamos.

VI-E. El solver también repara: 16 de 19 lecturas, sin errores silenciosos

Rechazar es honesto, pero no ayuda a quien sacó la foto. Sobre las 19 lecturas reales sin solución de la Tabla V, la reparación de la Sección V tarda menos de 200 ms por tablero. Devuelve la solución verdadera en 16, se abstiene en 3 y no entrega ninguna solución errónea. Casi todas las correcciones deshacen el fallo dominante, un $6+$ que era $5+$, y algunas cambian tres jaulas a la vez.

La regla que evita los errores silenciosos salió de un error. La primera versión aceptaba la corrección mínima si era la única de su tamaño. En un 4×4 con dos $5+$ leídos como $6+$, corrigió solo uno: el tablero cerraba, pero con otra solución. Ahora la corrección se acepta solo si ninguna otra, con los mismos cambios o con uno más, lleva a un tablero distinto; si no, el nodo se abstiene. Las 3 abstenciones son de ese tipo: dos explicaciones casi igual de plausibles y ninguna base para elegir.

El caso de la Fig. 4c es una foto tomada con el celular a la pantalla que mostraba la Fig. 2a. Los puntos del $\div$ se fundieron con la barra, la visión leyó los dos $3\div$ como $3+$ y el tablero quedó sin solución. El solver probó las alternativas y encontró que cambiar esas dos jaulas a $\div$ es la única corrección que lo cierra. La app lo dice y marca las jaulas corregidas.

VI-F. Las restricciones globales hacen el trabajo del solver

Para medir qué aporta cada decisión del modelo, resolvimos los 70 tableros reales con tres formulaciones (Tabla VI): la

Tabla VI

LAS RESTRICCIONES GLOBALES HACEN EL TRABAJO: CON ALLDIFFERENT NINGÚN TABLERO REAL REQUIERE RAMIFICAR; CON DESIGUALDADES BINARIAS LOS 9×9 NECESITAN HASTA 5 883 RAMAS Y TARDAN DIEZ VECES MÁS. VERDAD DE LOS 70 TABLEROS, UN HILO

Formulación	n	Mediana (ms)	Ramas (máx.)	Sin ramificar
Global (este trabajo)	4	0.8	0	47/47
	6	1.8	0	16/16
	9	7.7	0	7/7
Binaria ($x \neq y$)	4	1.4	0	47/47
	6	6.4	1 251	11/16
	9	81.9	5 883	1/7
Reificada ($-$, $\div$)	4	1.0	0	47/47
	6	1.8	0	16/16
	9	6.9	0	7/7

Tabla VII

EL COSTO CRECE CON n PERO SE QUEDA EN MILLISEGUNDOS: NINGÚN TABLERO REAL TARDA MÁS DE 14.1 MS EN RESOLVERSE. TIEMPO DE RESOLVER SOBRE LA VERDAD DE LOS 70 TABLEROS REALES (MS)

n	Tableros	Mediana	Máximo
4	47	4.4	7.8
6	16	4.1	5.1
9	7	10.8	14.1

nuestra; una *binaria*, que reemplaza cada AllDifferent por sus $n(n-1)/2$ desigualdades $x \neq y$; y una *reificada*, que modela resta y división con la disyunción de la Sección V. Usamos un solo hilo para que el número de ramas sea comparable. Con AllDifferent ningún tablero requiere ramificar: la propagación en la raíz, junto con el preprocesamiento de CP-SAT, fija las n^2 celdas. Las desigualdades binarias expresan lo mismo, pero propagan menos. En 9×9 , seis de siete tableros necesitan búsqueda, con hasta 5 883 ramas y 603 conflictos, y la mediana sube de 7.7 a 81.9 ms, diez veces más.

La propagación también sostiene el papel de verificador: con AllDifferent, las 19 lecturas sin solución de la Tabla V se refutan igualmente sin ramificar. La formulación reificada, en cambio, rinde lo mismo que el par máx / mín (6.9 frente a 7.7 ms de mediana en 9×9 , sin ramas en ambos casos). En KenKen, la reificación no cuesta ni aporta.

VI-G. El solver no es el cuello de botella

Medimos el tiempo de pared de `resolver`, incluida la construcción del modelo, en un Intel Core Ultra 7 255U (14 hilos) con OR-Tools 9.15 y Python 3.13. Sobre los 70 tableros reales (Tabla VII), la mediana no pasa de 10.8 ms en ningún tamaño. Sobre lo que el sistema lee, el máximo es de 20.4 ms, incluidas las instancias sin solución, en las que el solver tiene que agotar la búsqueda para demostrarlo. Para cubrir tamaños que los cuadernillos no traen, medimos además 30 instancias sintéticas por tamaño, de $n = 4$ a 9. Como no garantizan unicidad, miden el solver y no la dificultad de un puzzle publicado. Ninguna llega a 25 ms (mediana de 12.3 ms y máximo de 24.0 ms en 9×9).

VII. INTEGRACIÓN: DE LA FOTO A LA SOLUCIÓN SIN INTERVENCIÓN

Un CLI (`solve.py`) encadena las tres fases sin intervención manual: lee la imagen, construye y valida la instancia, la resuelve y dibuja la solución sobre una rejilla limpia (Fig. 2). Si el solver no encuentra solución, lo informa y dibuja el tablero sin rellenar: no inventa una.

La app móvil (Fig. 4) cierra el flujo para una persona con un KenKen en papel. Envía la foto en JSON a un servicio FastAPI en la PC (`kenken_nodo`, POST `/resolver`) y recibe el estado, las jaulas leídas, la solución, las correcciones y un mensaje en llano (Fig. 1). La conexión pasa por `adb reverse`, por USB o por depuración Wi-Fi, sin abrir puertos en la PC. En Modo Asistencia, el modo por defecto, el tablero aparece sin números y la persona revela solo las casillas que quiere; en Modo Rápido ve la solución completa, sobre una rejilla limpia o dibujada encima de su propia foto: el nodo devuelve dónde cae cada celda en la imagen, proyectada con la inversa de la homografía (Fig. 3). Cuando el nodo repara una lectura, la app lo dice y marca las jaulas corregidas; cuando se abstiene, muestra lo que leyó y pide otra foto. La ejecutamos en un teléfono Pixel; su usabilidad aún no está evaluada.

VIII. CONCLUSIONES

Separar percepción y razonamiento con un contrato explícito resolvió el puzzle y, además, nos dio un verificador. La visión sin aprendizaje lee el 97.09 % de 895 etiquetas reales, y su error se concentra en un solo par de glifos. El modelo de CP resuelve cualquier tablero real en milisegundos y, gracias a sus restricciones globales, sin ramificar. Contar soluciones bastó para que ninguno de los 22 tableros mal leídos llegara a una solución errónea, y las restricciones reificadas repararon 16 de las 19 lecturas sin solución. De paso, el solver encontró un error en nuestra propia verdad. El conjunto sintético, en cambio, resultó ser un mal juez: sirvió para desarrollar, pero las cifras que valen son las de los tableros reales y, entre ellas, las de los cuadernillos que no usamos al construir el sistema.

IX. LIMITACIONES Y TRABAJO FUTURO

Cinco limitaciones acotan estos resultados. (1) Los 70 tableros reales son de un mismo editor y una misma tipografía, Helvetica, de la que el banco de plantillas tiene un clon (FreeSans). Estas cifras no miden tipografías desconocidas, donde el conjunto sintético ya mostró caídas fuertes. (2) Son PDF rasterizados: tinta real, pero sin sombras, curvatura de papel ni desenfoque. Las condiciones fotográficas solo las cubre el conjunto sintético. (3) La ausencia de soluciones erróneas es un resultado sobre 22 tableros mal leídos, no una garantía. Una lectura equivocada que, por azar, deje un puzzle válido con solución única pasaría el filtro. (4) La reparación corrige la mayoría de las confusiones entre 5 y 6, pero las etiquetas de cuatro cifras que desbordan su celda se vuelven ilegibles y se rechazan antes de llegar al solver. (5) Las 16 reparaciones correctas son un resultado sobre 19 tableros de un solo editor.

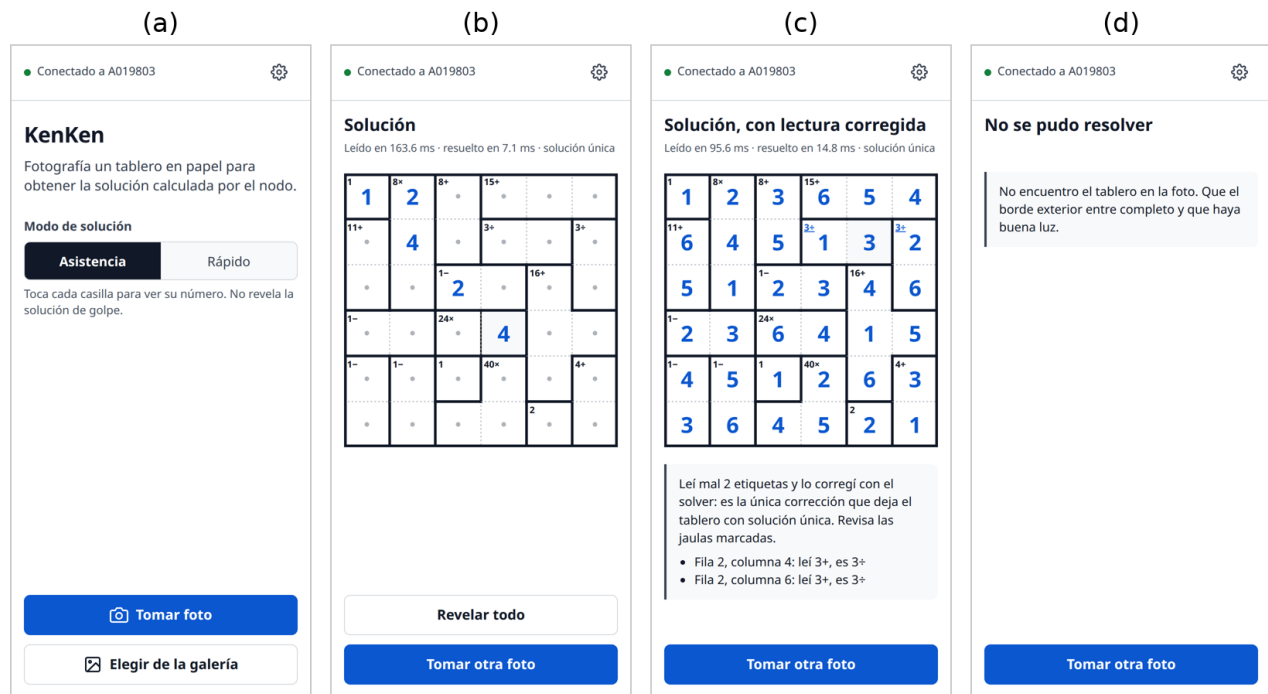


Figura 4. La app muestra lo que el solver decidió, incluida la corrección. (a) Inicio: estado del nodo y modo. (b) Modo Asistencia: el tablero sin números; la persona revela solo las casillas que toca. (c) Una foto de la Fig. 2a tomada a la pantalla: la visión leyó los dos 3÷ como 3+, el solver reparó la lectura y la app marca las jaulas corregidas. (d) Una foto cortada: el nodo no encuentra el tablero y lo dice en llano.

La reparación actual solo considera confusiones de un glifo y lecturas sin solución. Falta extenderla en tres direcciones: a las etiquetas ilegibles, que hoy se rechazan antes del solver; a las lecturas con varias soluciones; y a ponderar las candidatas con la confianza del clasificador en vez de con frecuencias fijas, que es la inferencia conjunta de Mulamba *et al.* [1].

DISPONIBILIDAD DE CÓDIGO Y DATOS

El código está en <https://github.com/ROSEWT/cs-topics>, en `labs/tbl-kenken`, con instrucciones de instalación y ejecución en su `README.md`. Cada cifra sale de un script: `scripts/punta_a_punta_scraped.py` (Tablas V y VII), `scripts/benchmark_ocr.py` (Tabla I), `scripts/etiquetas_por_split.py` y `scripts/e2e_por_split.py` (Tabla II), `scripts/comparar_ocr.py` y `scripts/evaluar_real.py` (Tabla III), `scripts/evaluar.py` 80 (Tabla IV), `scripts/comparar_formulaciones.py` (Tabla VI), `scripts/medir_reparacion.py` (Sección VI-E, con `kenken_cp/reparar.py`), `scripts/medir_solver.py` (tiempos sintéticos) y `scripts/figura_informe.py` (Fig. 2). El servicio del nodo está en `kenken_nodo` y la app en `app/` y `movil/`. Los cuadernillos de KrazyDad no se redistribuyen; `scripts/scrapear_kenken.py` los descarga y `scripts/extraer_dataset.py` reconstruye los tableros y su verdad.

DECLARACIÓN DE USO DE IA

El andamiaje de la Fase 1 (visión) se desarrolló con asistencia de IA generativa (Claude Code, de Anthropic). El modelo de Programación con Restricciones de la Fase 2 lo escribió el equipo, sin asistencia de IA. Con asistencia de Claude Code hicimos la redacción y compilación de este informe, la verificación de sus referencias y los scripts de medición, de comparación de formulaciones, de evaluación de punta a punta y de las figuras. Un integrante usó Gemini, de Google, para analizar la salida del benchmark de OCR. Todas las cifras salen de scripts del repositorio, se pueden volver a obtener y el equipo las revisó.

REFERENCIAS

- [1] M. Mulamba, J. Mandi, A. . Mahmutoullar y T. Guns, "Perception-based constraint solving for sudoku images," *Constraints*, vol. 29, no. 1–2, pp. 112–151, 2024, doi: 10.1007/s10601-024-09372-9.
- [2] M. Mulamba, J. Mandi, R. Canoy y T. Guns, "Hybrid classification and reasoning for image-based constraint solving," en *Proc. CPAIOR*, LNCS 12296, 2020, pp. 364–380, doi: 10.1007/978-3-030-58942-4_24.
- [3] KU Leuven AI, "Sudoku Assistant," 8 de febrero de 2023. [En línea]. Disponible: <https://ai.kuleuven.be/stories/post/2023-02-08-sudoku/>
- [4] P.-W. Wang, P. L. Donti, B. Wilder y J. Z. Kolter, "SATNet: Bridging deep learning and logical reasoning using a differentiable satisfiability solver," en *Proc. ICML*, 2019, pp. 6545–6554.
- [5] O. Chang, L. Flokas, H. Lipson y M. Spranger, "Assessing SATNet's ability to solve the symbol grounding problem," en *Proc. NeurIPS*, 2020.
- [6] J. P. Lewis, "Fast normalized cross-correlation," en *Proc. Vision Interface*, 1995, pp. 120–123.
- [7] P. Simha, K. Suraj y T. Ahobala, "Recognition of numbers and position using image processing techniques for solving Sudoku puzzles," en *Proc. ICAESM*, IEEE, 2012, pp. 1–5.

- [8] KrazyDad, “Inkies” (cuadernillos 4×4 , 6×6 y 9×9 , Volume 1, Book 1), KrazyDad.com. [En línea]. Disponible: <https://krazydad.com/inkies/>. Consultado: 18 de septiembre de 2026.
- [9] G. Bradski, “The OpenCV Library,” *Dr. Dobb’s Journal of Software Tools*, 2000.
- [10] N. Otsu, “A threshold selection method from gray-level histograms,” *IEEE Trans. Syst., Man, Cybern.*, vol. 9, no. 1, pp. 62–66, 1979.
- [11] L. Perron, F. Didier y S. Gay, “The CP-SAT-LP solver,” en *Proc. 29th Int. Conf. Principles and Practice of Constraint Programming (CP 2023)*, LIPIcs vol. 280, 2023, pp. 3:1–3:2, doi: 10.4230/LIPIcs.CP.2023.3.
- [12] OEIS Foundation, “A002860: Number of Latin squares of order n,” *The On-Line Encyclopedia of Integer Sequences*. [En línea]. Disponible: <https://oeis.org/A002860>
- [13] C. J. Colbourn, “The complexity of completing partial Latin squares,” *Discrete Applied Mathematics*, vol. 8, no. 1, pp. 25–30, 1984, doi: 10.1016/0166-218X(84)90075-1.
- [14] K. Haraguchi y H. Ono, “BlockSum is NP-complete,” *IEICE Trans. Inf. Syst.*, vol. E96-D, no. 3, pp. 481–488, 2013, doi: 10.1587/transinf.E96.D.481.
- [15] J. Cohen, “A coefficient of agreement for nominal scales,” *Educational and Psychological Measurement*, vol. 20, no. 1, pp. 37–46, 1960.
- [16] J. R. Landis y G. G. Koch, “The measurement of observer agreement for categorical data,” *Biometrics*, vol. 33, no. 1, pp. 159–174, 1977.
- [17] R. Smith, “An overview of the Tesseract OCR engine,” en *Proc. 9th Int. Conf. Document Analysis and Recognition (ICDAR)*, 2007, pp. 629–633.
- [18] Y. Du *et al.*, “PP-OCR: A practical ultra lightweight OCR system,” arXiv:2009.09941, 2020.
- [19] M. Li *et al.*, “TrOCR: Transformer-based optical character recognition with pre-trained models,” en *Proc. AAAI Conf. Artificial Intelligence*, 2023.
- [20] Q. McNemar, “Note on the sampling error of the difference between correlated proportions or percentages,” *Psychometrika*, vol. 12, no. 2, pp. 153–157, 1947.